

# System Design — Batch 1: Distributed Coordination & Scaling

---

Topics: 2-Phase Commit (2PC) | Saga Pattern | Stateless vs Stateful Services | Rebalancing & Resharding

---

## Topic 1: 2-Phase Commit (2PC)

---

### 1. Intuition

Imagine you and three friends are planning to split the bill at a restaurant. Before anyone pays, one person (the coordinator) asks everyone: "Hey, do you all have enough money to pay your share?"

- Friend A checks wallet: "Yes, I'm good." (VOTE COMMIT)
- Friend B checks wallet: "Yes, I'm good." (VOTE COMMIT)
- Friend C checks wallet: "...I forgot my wallet." (VOTE ABORT)

Since Friend C can't commit, the coordinator says: "Nobody pay. We're rolling back. Let's figure something else out."

If **all** had said yes, the coordinator would say: "Everyone pay now." — and everyone commits.

The key thing: **nobody pays until everyone confirms they CAN pay**. The coordinator holds everyone hostage until there's unanimous agreement. That's 2PC.

---

### 2. Reality

**Definition:** 2PC is a distributed consensus protocol that ensures **all participants in a distributed transaction either commit or abort together** — achieving atomicity across multiple nodes.

**Where it's used:**

- Traditional distributed databases (MySQL Cluster, PostgreSQL with 2PC)
- XA transactions (Java EE / JTA)
- Cross-database transactions in monolithic systems
- Google Spanner (a modified version with TrueTime)

**Pros:**

- Guarantees **strong consistency** — ACID across distributed nodes
- Simple to reason about — binary outcome: all commit or all abort

- Well-understood protocol, decades of battle-testing

#### Cons:

- **Blocking protocol** — if the coordinator crashes after Phase 1 but before Phase 2, all participants are stuck holding locks, waiting. This is the Achilles' heel.
  - **Latency killer** — requires multiple network round trips (prepare + commit), all synchronous
  - **Doesn't scale horizontally** — the coordinator is a single point of failure and a bottleneck
  - **Lock contention** — participants hold resource locks throughout both phases
- 

## 3. How It Works

There are **3 actors**: a **Coordinator** (the one driving the transaction) and **N Participants** (the databases/services involved).

### Phase 1: PREPARE (The Voting Phase)

1. **Coordinator starts** — it assigns a **global transaction ID (TxID)** and sends a `PREPARE` message to every participant.
2. **Each participant receives `PREPARE` and does the following locally:**
  - Executes the entire transaction **locally** (e.g., `UPDATE balance = balance - 100`)
  - Does **NOT** commit yet — the changes are in a temporary state
  - Writes all changes to its **Write-Ahead Log (WAL)** on disk — this is critical. Even if the participant crashes, it can recover from the WAL and still honor its vote.
  - **Acquires all locks** needed for this transaction (row locks, table locks, whatever is required). These locks are **held open** — nobody else can touch these rows.
  - Responds to the coordinator: either `__CODE0` (*I've done the work, written to WAL, holding locks, ready to commit*) or `CODE1__` (something went wrong — constraint violation, disk full, timeout)
3. **Coordinator collects all votes.**

### Phase 2: COMMIT or ABORT (The Decision Phase)

If **ALL** participants voted YES:

1. Coordinator writes `COMMIT` decision to **its own WAL** first (this is the **point of no return** — the decision is now durable)
2. Coordinator sends `COMMIT` to all participants
3. Each participant:
  - Commits the transaction locally (makes changes permanent)
  - **Releases all locks**
  - Sends `ACK` back to coordinator
4. Coordinator receives all ACKs → transaction is complete. Cleans up.

### If ANY participant voted NO (or timed out):

1. Coordinator writes `ABORT` to its own WAL
2. Coordinator sends `ABORT` to all participants
3. Each participant:
  - Rolls back using its WAL
  - Releases all locks
  - Sends `ACK`
4. Done. Nothing was committed anywhere.

## The Full Flow (Visual)

### Phase 1: PREPARE (Voting Phase)

```
Coordinator → Participant A: "Can you commit?"
Coordinator → Participant B: "Can you commit?"
Coordinator → Participant C: "Can you commit?"
```

#### Each participant:

1. Executes the transaction locally (but does NOT commit)
2. Writes changes to a WAL (Write-Ahead Log) for durability
3. Acquires all necessary locks
4. Responds: YES (I can commit) or NO (I can't)

### Phase 2: COMMIT / ABORT (Decision Phase)

#### IF all participants voted YES:

```
Coordinator → All: "COMMIT"
Each participant commits locally, releases locks.
```

#### IF any participant voted NO (or timed out):

```
Coordinator → All: "ABORT"
Each participant rolls back, releases locks.
```

## The Dangerous In-Doubt Window

Here's where it gets ugly. Consider this timeline:

```
Time 0: Coordinator sends PREPARE to A, B, C
Time 1: A votes YES, B votes YES, C votes YES
Time 2: Coordinator receives all YES votes
Time 3: Coordinator writes COMMIT to its WAL
Time 4: Coordinator sends COMMIT to A ✓
Time 5: ⚡ Coordinator CRASHES before sending COMMIT to B and C
```

### Now **B** and **C** are stuck:

- They voted YES → they're holding locks
- They wrote to WAL → they're ready to commit
- But they **never received the final COMMIT or ABORT** decision
- They **cannot decide on their own** — what if A got ABORT? They don't know.

~~They cannot release locks — what if the decision was COMMIT? They need to honor it.~~

- They just **wait**. Holding locks. Blocking other transactions.

This is the **blocking problem** of 2PC. B and C remain in this **in-doubt state** until the coordinator recovers, reads its WAL (which says COMMIT), and re-sends the decision.

If the coordinator's disk is also dead? You need manual intervention or a recovery protocol that contacts other participants to learn the decision (A knows it was COMMIT because it received it).

## 4. When to Rely on It

In System Design, 2PC comes up when:

- You **absolutely need strong consistency** across multiple data stores
- The operation is **low-throughput, high-value** (e.g., financial settlement between two banks)
- You're designing within a **single datacenter** (latency is manageable)
- The interviewer asks: "How do you ensure atomicity across your Order DB and Payment DB?" — 2PC is the textbook starting point (before you explain why you'd likely move away from it)

## 5. Trade-offs & Common Mistakes

| Mistake                                          | Why it's wrong                                                                                  |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Using 2PC across microservices over the internet | Latency + coordinator failure = disaster. Use Sagas instead.                                    |
| Assuming coordinator failure is rare             | At scale, it's not. You need coordinator recovery + participant timeout logic.                  |
| Holding locks for too long                       | If Phase 1 involves a slow participant, everyone waits. One slow node kills throughput for all. |
| Using 2PC across regions                         | Cross-region RTT (50-200ms) makes the blocking window enormous.                                 |
| Ignoring the "in-doubt" state                    | If you can't handle participants stuck waiting, your system will deadlock under failure.        |

**When NOT to use it:** High-throughput systems, cross-region deployments, microservices architectures, anything where availability matters more than strict consistency.

## 6. Key Numbers

| Metric | Value |
|--------|-------|
|--------|-------|

|                                                        |                                |
|--------------------------------------------------------|--------------------------------|
| Minimum round trips                                    | 2 (prepare + commit)           |
| Same-datacenter 2PC latency                            | ~5-20ms                        |
| Cross-region 2PC latency                               | ~100-500ms (brutal)            |
| Lock hold duration                                     | Entire duration of both phases |
| Google Spanner (modified 2PC + TrueTime) write latency | ~10-15ms within a region       |

## 7. Scenario

**Inter-bank fund transfer system.** Bank A debits \$1000, Bank B credits \$1000. This **MUST** be atomic. You cannot debit without crediting. Both banks are within the same settlement network (low latency). Transaction volume: ~1000 TPS, not millions. 2PC fits here — strong consistency, manageable scale, single-region.

## 8. Practical Question

**Q:** You're designing an e-commerce checkout system. When a user places an order, you need to: (1) deduct inventory in the Inventory Service, (2) charge payment in the Payment Service, (3) create the order in the Order Service. A senior engineer suggests using 2PC. Do you agree?

**A:** No, not for this design. Here's why:

These are three independent **microservices**, likely deployed across different nodes, possibly different regions. 2PC would:

- Create a **single coordinator as a bottleneck** for every checkout
- **Block all three services** if the coordinator fails mid-transaction
- **Kill throughput** — every checkout holds locks across three services simultaneously
- Violate the microservice principle of **independent deployability and failure isolation**

The better approach: use the **Saga pattern** — specifically a choreography-based saga where each service emits events, and compensating transactions handle failures. If Payment fails after Inventory was deducted, a compensating transaction restores the inventory.

The exception: if all three were tables in a **single database**, then a local ACID transaction (not even 2PC) would suffice. 2PC only makes sense if you have exactly 2-3 databases in the same datacenter with low throughput and hard consistency requirements.

## Topic 2: Saga Pattern

# 1. Intuition

Think of planning a vacation with multiple bookings: flight, hotel, car rental.

You book the flight first — confirmed. Then you book the hotel — confirmed. Then you try to book the car rental — **declined, no cars available**.

Now what? You don't just accept a partial vacation. You **undo** in reverse order:

1. Cancel the hotel booking (compensating transaction)
2. Cancel the flight booking (compensating transaction)

Nobody coordinated all three with a single lock. Each booking was an independent transaction. When one failed, you **compensated** backwards. That's a Saga.

The critical insight: **each step commits independently**. There's no global lock. If something fails midway, you don't "rollback" — you execute **compensating actions** to logically undo what was already committed.

## 2. Reality

**Definition:** A Saga is a sequence of **local transactions** where each transaction updates a single service/database and publishes an event or triggers the next step. If any step fails, previously completed steps are **compensated** (undone) by executing reverse operations.

**Coined by:** Hector Garcia-Molina and Kenneth Salem, 1987 paper — originally for long-lived database transactions.

**Two flavors:**

|                                | Choreography                                 | Orchestration                                        |
|--------------------------------|----------------------------------------------|------------------------------------------------------|
| <b>How</b>                     | Each service listens for events and reacts   | A central orchestrator tells each service what to do |
| <b>Coupling</b>                | Loose — services don't know about each other | Tighter — orchestrator knows the full workflow       |
| <b>Complexity</b>              | Simple for 2-3 steps, spaghetti for 5+       | Cleaner for complex multi-step workflows             |
| <b>Debugging</b>               | Hard — flow is implicit in event chains      | Easier — flow is explicit in the orchestrator        |
| <b>Single point of failure</b> | None                                         | Orchestrator (must be made resilient)                |

**Pros:**

- No global locks — each service commits independently → **high throughput**
- Works across microservices, across regions
- Maintains **high availability** — no blocking
- Natural fit for event-driven architectures

- **Eventual consistency** — between steps, data is temporarily inconsistent
- Compensating transactions are **hard to write correctly** (e.g., how do you "un-send" an email?)
- Debugging distributed sagas is painful without good observability
- **No isolation** — other transactions can see intermediate states (dirty reads)

## 3. How It Works

### 3A. Choreography-Based Saga

#### Happy Path (Step by Step)

##### Step 1: Order Service

- Receives user's "place order" request
- Creates order in its own DB (status: PENDING)
- COMMITS locally (this is a real, permanent commit)
- Publishes event: OrderCreated {orderId: 123, items: [...], userId: 456}
- Done. Order Service's job is over (for now).

##### Step 2: Inventory Service

- Subscribed to "OrderCreated" events on the message broker (Kafka/RabbitMQ)
- Picks up the event
- Checks stock → sufficient
- Reserves the items in its own DB (decrements available\_qty)
- COMMITS locally
- Publishes event: InventoryReserved {orderId: 123, reservationId: 789}

##### Step 3: Payment Service

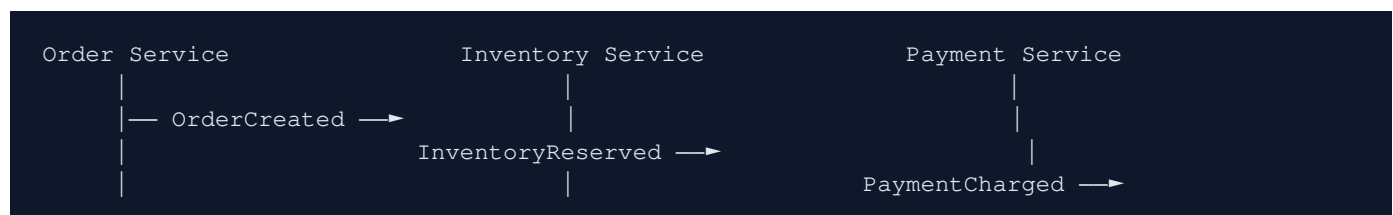
- Subscribed to "InventoryReserved" events
- Picks up the event
- Charges the customer's payment method
- COMMITS locally
- Publishes event: PaymentCharged {orderId: 123, paymentId: 012}

##### Step 4: Order Service (again)

- Subscribed to "PaymentCharged" events
- Picks up the event
- Updates order status: PENDING → CONFIRMED
- COMMITS locally
- Done. User sees "Order Confirmed."

**Key mechanic:** Every step is a **fully committed local transaction**. There's no global lock, no prepare phase, no coordinator. Services communicate purely through events on a message broker.

#### Visual Flow



| ← OrderConfirmed ————— |

## Failure Path (Compensation)

Say Payment fails at Step 3:

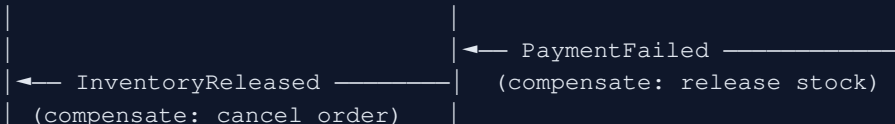
```
Step 3 (failed): Payment Service
- Picks up InventoryReserved event
- Tries to charge customer → DECLINED (insufficient funds)
- Publishes event: PaymentFailed {orderId: 123, reason: "insufficient_funds"}
```

```
Compensation Step 2: Inventory Service
- Subscribed to "PaymentFailed" events
- Picks up the event
- Runs compensating transaction: releases the reserved items
  (UPDATE inventory SET available_qty = available_qty + reserved_qty
   WHERE reservation_id = 789)
- COMMITS locally
- Publishes event: InventoryReleased {orderId: 123}
```

```
Compensation Step 1: Order Service
- Subscribed to "InventoryReleased" events (after failure)
- Picks up the event
- Runs compensating transaction: marks order as FAILED
  (UPDATE orders SET status = 'FAILED' WHERE id = 123)
- COMMITS locally
- User sees "Order Failed - payment declined."
```

## Visual Flow (Failure)

ON FAILURE (Payment fails):



## How Each Service Knows the Flow

It doesn't know the **full** flow. Each service only knows:

- "When I see event X, I do my work and emit event Y"
- "When I see failure event Z, I run my compensation"

This is the **strength** (loose coupling) and the **weakness** (nobody has the full picture; debugging a 7-service chain by tracing events across Kafka topics is painful).

## Critical Mechanic: Transactional Outbox

There's a subtle problem. Consider Step 1:

```
Order Service:
1. INSERT into orders table (status: PENDING) → DB operation
2. Publish OrderCreated to Kafka → Broker operation
```

What if the DB write succeeds but the Kafka publish fails (network blip)? Now you have an order in the DB



but no event was emitted — the saga is stuck forever.

The solution is the **Transactional Outbox Pattern**:

```
Order Service:
1. BEGIN TRANSACTION
2.   INSERT into orders (status: PENDING)
3.   INSERT into outbox_table (event: OrderCreated, payload: {...})
4. COMMIT ← single atomic DB transaction

Separately, a poller/CDC process:
5. Reads outbox_table
6. Publishes event to Kafka
7. Marks outbox row as published
```

Now the event and the data change are in the **same DB transaction** — atomic. A background process reliably pushes events to the broker. This is how production choreography sagas actually work.

### 3B. Orchestration-Based Saga

There **is** a central brain: the **Saga Orchestrator**. Think of it like a project manager with a checklist — it tells each service exactly what to do, waits for the response, and decides the next step.

#### The Foundation: A Persisted State Machine

Before any saga runs, you define the entire workflow as a **state machine** and store it in a durable database:

```
States:
STARTED
ORDER_CREATED
INVENTORY_RESERVED
PAYMENT_CHARGED
COMPLETED
— failure states —
PAYMENT_FAILED
COMPENSATING_INVENTORY
COMPENSATING_ORDER
FAILED

Transitions (forward):
STARTED           —[create order]→      ORDER_CREATED
ORDER_CREATED     —[reserve inventory]→  INVENTORY_RESERVED
INVENTORY_RESERVED —[charge payment]→    PAYMENT_CHARGED
PAYMENT_CHARGED   —[finalize]→           COMPLETED

Transitions (compensation):
PAYMENT_FAILED    —[release inventory]→  COMPENSATING_INVENTORY
COMPENSATING_INVENTORY —[cancel order]→  COMPENSATING_ORDER
COMPENSATING_ORDER —[done]→             FAILED
```

Every single state transition is **written to the database before the next step executes**. This is the entire recovery story.

#### Happy Path (Step by Step)

User calls: POST /orders {items: [...], userId: 456}

Step 1: Orchestrator boots up

- Creates a saga instance row in its database:  
INSERT INTO sagas (saga\_id, state, payload, created\_at)  
VALUES ('abc-123', 'STARTED', '{items: [...], userId: 456}', NOW())

Step 2: Orchestrator sends a COMMAND to Order Service

- This is a directed command (not a broadcast event)
- Goes through a message queue (e.g., Kafka topic `order-commands` or direct gRPC call)
- Message: {saga\_id: "abc-123", command: "CREATE\_ORDER", data: {userId: 456, items: [...]}}
- Order Service receives it, does its work:  
INSERT INTO orders (id, user\_id, status) VALUES (123, 456, 'PENDING')  
COMMIT
- Order Service replies to the orchestrator (via reply queue or callback):  
{saga\_id: "abc-123", status: "SUCCESS", orderId: 123}

Step 3: Orchestrator updates its state

- UPDATE sagas SET state = 'ORDER\_CREATED', context = '{"orderId": 123}'  
WHERE saga\_id = 'abc-123'
- Consults state machine: "I'm at ORDER\_CREATED. Next transition: reserve inventory."

Step 4: Orchestrator sends a COMMAND to Inventory Service

- Message: {saga\_id: "abc-123", command: "RESERVE\_INVENTORY", data: {orderId: 123, items: [...]}}
- Inventory Service receives it:  
UPDATE inventory SET reserved\_qty = reserved\_qty + 2 WHERE item\_id = 'ITEM-A'  
INSERT INTO reservations (id, order\_id, item\_id, qty) VALUES (789, 123, 'ITEM-A', 2)  
COMMIT
- Replies: {saga\_id: "abc-123", status: "SUCCESS", reservationId: 789}

Step 5: Orchestrator updates state

- UPDATE sagas SET state = 'INVENTORY\_RESERVED',  
context = '{"orderId": 123, "reservationId": 789}'  
WHERE saga\_id = 'abc-123'

Step 6: Orchestrator sends a COMMAND to Payment Service

- Message: {saga\_id: "abc-123", command: "CHARGE\_PAYMENT", data: {userId: 456, amount: 99.99, orderId: 123}}
- Payment Service charges the card:  
INSERT INTO payments (id, order\_id, amount, status) VALUES (012, 123, 99.99, 'CHARGED')  
COMMIT
- Replies: {saga\_id: "abc-123", status: "SUCCESS", paymentId: 012}

Step 7: Orchestrator finalizes

- UPDATE sagas SET state = 'COMPLETED' WHERE saga\_id = 'abc-123'
- Returns 201 Created to the user. Order confirmed.

The full DB trail looks like this:

| saga_id | state              | timestamp    |
|---------|--------------------|--------------|
| abc-123 | STARTED            | 10:00:00.000 |
| abc-123 | ORDER_CREATED      | 10:00:00.045 |
| abc-123 | INVENTORY_RESERVED | 10:00:00.092 |
| abc-123 | PAYMENT_CHARGED    | 10:00:00.188 |

|         |           |              |
|---------|-----------|--------------|
| abc-123 | COMPLETED | 10:00:00.190 |
|---------|-----------|--------------|

## Failure Path (Compensation in Action)

Same flow, but Payment fails at Step 6:

```
Steps 1-5: Identical. Order created, inventory reserved. State = INVENTORY_RESERVED.

Step 6: Orchestrator sends COMMAND to Payment Service
- Payment Service tries to charge → card declined
- Replies: {saga_id: "abc-123", status: "FAILED", reason: "card_declined"}

Step 7: Orchestrator receives failure, updates state
- UPDATE sagas SET state = 'PAYMENT_FAILED' WHERE saga_id = 'abc-123'
- Consults state machine: "I'm at PAYMENT_FAILED.
  Compensation path: first release inventory, then cancel order."

Step 8: Orchestrator sends COMPENSATE command to Inventory Service
- Message: {saga_id: "abc-123", command: "RELEASE_INVENTORY", data: {reservationId: 789}}
- Inventory Service:
  UPDATE inventory SET reserved_qty = reserved_qty - 2 WHERE item_id = 'ITEM-A'
  DELETE FROM reservations WHERE id = 789
  COMMIT
- Replies: {saga_id: "abc-123", status: "SUCCESS"}

Step 9: Orchestrator updates state
- UPDATE sagas SET state = 'COMPENSATING_INVENTORY' WHERE saga_id = 'abc-123'

Step 10: Orchestrator sends COMPENSATE command to Order Service
- Message: {saga_id: "abc-123", command: "CANCEL_ORDER", data: {orderId: 123}}
- Order Service:
  UPDATE orders SET status = 'CANCELLED' WHERE id = 123
  COMMIT
- Replies: {saga_id: "abc-123", status: "SUCCESS"}

Step 11: Orchestrator finalizes failure
- UPDATE sagas SET state = 'FAILED' WHERE saga_id = 'abc-123'
- Returns 400 / 422 to user: "Payment declined. Order cancelled."
```

## The DB trail for the failure case:

| saga_id | state                  | timestamp    |
|---------|------------------------|--------------|
| abc-123 | STARTED                | 10:00:00.000 |
| abc-123 | ORDER_CREATED          | 10:00:00.045 |
| abc-123 | INVENTORY_RESERVED     | 10:00:00.092 |
| abc-123 | PAYMENT_FAILED         | 10:00:00.190 |
| abc-123 | COMPENSATING_INVENTORY | 10:00:00.235 |
| abc-123 | COMPENSATING_ORDER     | 10:00:00.280 |
| abc-123 | FAILED                 | 10:00:00.282 |

## What If the Orchestrator Crashes?

**Scenario:** Orchestrator crashes between Step 5 (INVENTORY\_RESERVED saved) and Step 6 (payment command):

|              |                                 |
|--------------|---------------------------------|
| 10:00:00.092 | State saved: INVENTORY_RESERVED |
|--------------|---------------------------------|

```

10:00:00.093 ✂ Orchestrator process crashes

... Orchestrator restarts (new pod spins up) ...

Recovery Step 1:
- Orchestrator queries its DB:
  SELECT * FROM sagas WHERE state NOT IN ('COMPLETED', 'FAILED')
- Finds saga_id: 'abc-123', state: INVENTORY_RESERVED

Recovery Step 2:
- Consults state machine: "INVENTORY_RESERVED → next step: charge payment"

Recovery Step 3:
- Resumes from Step 6 — sends the payment command as if nothing happened

```

**This is why persisting state before each step matters.** The orchestrator is **stateless in memory** — all state lives in the DB. You can kill it, restart it, even run multiple orchestrator instances, and it picks up exactly where it left off.

**What if the crash happens AFTER sending the command but BEFORE saving the next state?**

```

10:00:00.092 State: INVENTORY_RESERVED
10:00:00.093 Sends CHARGE_PAYMENT to Payment Service ← sent!
10:00:00.094 ✂ Crash before saving PAYMENT_CHARGED

```

On restart, it sees state = INVENTORY\_RESERVED, so it sends the payment command **again**. This means Payment Service might receive the charge command **twice**. This is exactly why:

**Every saga participant MUST be idempotent.**

Payment Service must check: "Have I already processed a charge for saga *id abc-123*?" *If yes, return the cached result. If no, process it. Idempotency keys (saga *id* + step) solve this.*

## Choreography vs Orchestration — Mechanical Difference

| Aspect                 | Choreography                                   | Orchestration                               |
|------------------------|------------------------------------------------|---------------------------------------------|
| Communication          | Events (broadcast, reactive)                   | Commands (directed, imperative)             |
| Who decides next step? | Each service independently                     | The orchestrator                            |
| State tracking         | Implicit (scattered across service DBs)        | Explicit (saga state machine in one DB)     |
| Flow visibility        | Must trace events across broker topics         | Read the orchestrator's state table         |
| Adding a new step      | Touch multiple services (who listens to what?) | Add one step in the orchestrator definition |
|                        |                                                |                                             |

|                  |                                                 |                                                              |
|------------------|-------------------------------------------------|--------------------------------------------------------------|
| Failure recovery | Each service must know its compensation trigger | Orchestrator drives all compensations from its state machine |
|------------------|-------------------------------------------------|--------------------------------------------------------------|

## Real-World Implementations

| Framework                 | How it works                                                                                                          |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Temporal</b>           | Orchestrator logic is your code. Temporal runtime persists every state transition. Replays your function on recovery. |
| <b>AWS Step Functions</b> | State machine defined in JSON (ASL). AWS manages persistence and retries.                                             |
| <b>Cadence</b> (Uber)     | Same concept as Temporal (Temporal is the successor).                                                                 |
| <b>Netflix Conductor</b>  | Workflow orchestrator with a JSON-defined DAG of tasks.                                                               |

## 4. When to Rely on It

Sagas show up in SD when:

- The interviewer says "how do you handle transactions across multiple microservices?"
- You've already split your system into independent services with their own databases (**database-per-service pattern**)
- You need **high throughput and availability** and can tolerate brief inconsistency
- The workflow has **clear compensating actions** for each step
- You're designing: order processing, booking systems, payment workflows, multi-step onboarding

**The moment in SD:** Right after you draw your microservice boundaries and the interviewer asks: "But what if Step 3 fails after Steps 1 and 2 succeeded?" — that's your Saga cue.

## 5. Trade-offs & Common Mistakes

| Mistake                                         | Why it's wrong                                                                                                                   |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Not designing compensating transactions upfront | You'll discover mid-incident that you can't undo Step 3. Design compensations at design time.                                    |
| Using choreography for 7+ step workflows        | Event chains become untraceable spaghetti. Switch to orchestration at ~4-5 steps.                                                |
| Ignoring idempotency                            | If a compensating event is retried (network issue), it must be safe to execute twice. Every saga step <b>MUST</b> be idempotent. |

|                                                     |                                                                                                                                        |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Expecting isolation                                 | Other reads can see "order created but not yet paid" — design your UI/API to handle intermediate states (e.g., "Order Processing..."). |
| Not persisting saga state                           | If the orchestrator crashes, you lose track of which step you're on. Persist the saga state machine to a durable store.                |
| Treating Saga as a replacement for all transactions | If your services share a single DB, just use a local ACID transaction. Don't over-engineer.                                            |

## 6. Key Numbers

| Metric                                          | Value                                          |
|-------------------------------------------------|------------------------------------------------|
| End-to-end saga latency (3 steps, same region)  | ~50-200ms                                      |
| End-to-end saga latency (3 steps, cross-region) | ~300-800ms                                     |
| Inconsistency window                            | Duration of the saga (milliseconds to seconds) |
| Typical saga step count in production           | 3-7 steps                                      |
| Message broker latency per hop (Kafka)          | ~2-10ms                                        |

## 7. Scenario

### E-commerce order processing at scale.

When a user places an order on Amazon-scale:

1. **Order Service** → creates order (status: PENDING)
2. **Inventory Service** → reserves items
3. **Payment Service** → charges the customer
4. **Shipping Service** → schedules delivery
5. **Notification Service** → sends confirmation email

This is 5 services, millions of orders/day. 2PC would be a death sentence. An **orchestration-based saga** with a durable state machine (backed by a DB or workflow engine like Temporal/Cadence) handles this cleanly. If payment fails at Step 3, compensate: release inventory (Step 2), cancel order (Step 1). Steps 4 and 5 never execute.

## 8. Practical Question

**Q:** You're designing a travel booking platform. A user books a trip that includes: Flight (via Airline API), Hotel (via Hotel API), and Car Rental (via Car API). All three must succeed, or none should.

These are **external third-party APIs** you don't control. How do you handle this?

**A:**

**Use an orchestration-based Saga.** Here's why and how:

**Why not 2PC?** You don't control the third-party APIs. They won't participate in your 2PC protocol. Non-starter.

**Design:**

```
Saga Orchestrator (state machine, persisted to DB)
|
Step 1: Book Flight → call Airline API → get confirmation_id
Step 2: Book Hotel → call Hotel API → get reservation_id
Step 3: Book Car → call Car API → ✗ FAILED (no availability)
|
Compensate Step 2: Cancel Hotel → call Hotel API cancel(reservation_id)
Compensate Step 1: Cancel Flight → call Airline API cancel(confirmation_id)
|
Return to user: "Trip booking failed — car unavailable. All bookings reversed."
```

**Critical design details:**

- **Store confirmation IDs** at each step — you need them for compensation
- **Idempotency keys** on every API call — if the network times out and you retry, you don't double-book
- **Timeout handling** — if Hotel API hangs for 30s, you need a timeout + compensate flight
- **Saga state persisted** — if your orchestrator crashes between Step 2 and Step 3, on recovery it reads state from DB and resumes
- **Dead letter queue** — if a compensating call fails (Airline API is down during cancel), push to DLQ and retry with backoff. Alert ops if compensation is stuck.

## 2PC vs Saga — The Cheat Sheet

| Dimension        | 2PC                           | Saga                            |
|------------------|-------------------------------|---------------------------------|
| Consistency      | Strong (ACID)                 | Eventual                        |
| Availability     | Lower (blocking)              | Higher (non-blocking)           |
| Latency          | Higher (lock + 2 round trips) | Lower per step (no global lock) |
| Scalability      | Poor (coordinator bottleneck) | Good (decentralized)            |
| Failure handling | Rollback                      | Compensating transactions       |
| Isolation        | Full                          | None (dirty reads possible)     |

|            |                                                          |                                                            |
|------------|----------------------------------------------------------|------------------------------------------------------------|
| Best for   | Single DC, few participants, strong consistency required | Microservices, cross-region, high throughput               |
| Avoid when | Cross-region, high TPS, microservices                    | You need strict isolation, or compensations are impossible |

## Topic 3: Stateless vs Stateful Services

### 1. Intuition

Think of two customer support desks:

- Desk A (stateless): every time a customer comes, the agent reads the ticket from a shared system, solves the issue, and forgets everything locally. Any agent can take the next request.
- Desk B (stateful): each agent keeps customer history in a personal notebook. If that agent is absent, no one else has the latest context.

In systems:

- Stateless service = request context lives outside the service instance.
- Stateful service = service instance holds durable or session-critical state that matters for correctness.

Simple mental model:

- Stateless is like rental cars: switch easily.
- Stateful is like your own car with your stuff inside: harder to swap.

### 2. Reality

#### Stateless Service

**Definition:** A service where any instance can handle any request because no client-specific durable state is stored in process memory or local disk for future requests.

**Common usages:**

- API gateways
- Auth token validators
- Web app backend tiers
- Read-only compute workers
- Microservices with DB as source of truth

**Pros:**

- Horizontal scaling is easy (add more replicas)



#### Simple load balancing

- Fast failover and auto-healing
- Easy rolling deployments
- Better elasticity for burst traffic

#### Cons:

- External state lookups add network latency
- More dependency on caches/DB/session stores
- Can increase infra cost if external stores are heavily used

## Stateful Service

**Definition:** A service that stores and depends on local or node-affined state across requests (session state, in-memory shard ownership, local logs, stream offsets, etc.).

#### Common usages:

- Databases
- Kafka brokers
- Redis primary nodes
- WebSocket/chat connection managers
- Game servers with room affinity
- Stream processors with local state stores

#### Pros:

- Can be very fast for repeated access to local state
- Good for workloads needing affinity or ordered processing
- Reduces repeated remote lookups for hot state

#### Cons:

- Harder scaling and rebalancing
- More complex failover and recovery
- Harder deployments and node replacement
- Higher operational complexity (replication, snapshot, restore)

## 3. How It Works

### How Stateless Works

```
Step 1: Client request arrives through load balancer
Step 2: LB picks any healthy instance (round-robin, least-connections, etc.)
Step 3: Instance reads needed state from external systems (DB/cache/object store)
Step 4: Instance computes and returns response
Step 5: Instance discards request context – nothing stored locally
```

## Scaling behavior:

Current: 3 instances handling 3000 QPS (1000 each)  
Traffic spikes to 6000 QPS

### Phase 1: AUTO-SCALE

- Autoscaler detects CPU/QPS threshold breached
- Spins up 3 new instances

### Phase 2: REGISTER

- New instances register with load balancer health check
- LB starts routing traffic to them

### Phase 3: SERVE

- 6 instances now handle 6000 QPS (1000 each)
- No data migration, no state transfer, no warmup needed
- Linear capacity gain until DB/cache becomes the bottleneck

### Phase 4: SCALE-DOWN (traffic drops)

- Autoscaler removes 3 instances
- In-flight requests on removed instances complete or drain
- No state to migrate - just terminate the pod

## Failure behavior:

### Instance 2 crashes:

- LB detects failed health check within ~5-10 seconds
- Removes Instance 2 from rotation
- All traffic redistributes to Instance 1 and Instance 3
- Zero data loss - no state was stored on Instance 2
- New Instance 4 can spin up and immediately serve traffic

## How Stateful Works

- Step 1: Request arrives - needs a specific node (ownership/partition/session affinity)
- Step 2: Router/LB sends request to the correct node based on partition key
- Step 3: Service reads/writes node-local or partition-local state
- Step 4: State changes replicated or checkpointed for durability
- Step 5: On failure, leader election/replay/recovery happens
- Step 6: Traffic may pause or degrade during failover/rebuild

## Scaling behavior:

Current: 4 Kafka brokers, 16 partitions (4 per broker)  
Need to add Broker 5

### Phase 1: PLAN

- Decide which partitions move to Broker 5
- Target: ~3 partitions per broker ( $16/5 \approx 3$ )

### Phase 2: REPLICATE

- Broker 5 starts replicating data for partitions being moved
- Existing brokers still serve traffic for those partitions
- Background data streaming: can take minutes to hours depending on data size

#### Phase 3: CATCH-UP

- Broker 5 replays recent writes to close the replication gap
- Gap shrinks to near-zero

#### Phase 4: CUTOVER

- Leadership for moved partitions transfers to Broker 5
- Producers/consumers now communicate with Broker 5 for those partitions

#### Phase 5: CLEANUP

- Old brokers delete replicas of moved partitions
- Rebalancing complete

Total time: minutes to hours (not seconds like stateless)

### Failure behavior:

Broker 3 dies (owned 4 partitions):

#### Phase 1: DETECT

- ZooKeeper/Controller detects Broker 3 is unresponsive (~10-30 seconds)

#### Phase 2: LEADER ELECTION

- For each of Broker 3's partitions, an in-sync replica (ISR) on another broker is promoted to leader
- Partition 7: Broker 1's replica → new leader
- Partition 8: Broker 2's replica → new leader
- Partition 9: Broker 4's replica → new leader
- Partition 10: Broker 1's replica → new leader

#### Phase 3: RECOVERY

- Producers/consumers get new leader info from metadata
- Traffic resumes for those partitions
- Partial unavailability window: ~seconds to a minute

#### Phase 4: REBALANCE

- If Broker 3 is permanently dead, replicate those partitions to surviving brokers for fault tolerance
- This is data movement - takes time

## 4. When to Rely on It in SD

Where it appears in interview flow — after you define APIs and rough architecture, interviewer asks:

- "How will this scale to 10x traffic?"
- "What happens during instance failure?"
- "Do you need sticky sessions?"

### Use stateless when:

- You need fast horizontal scale
- Low operational complexity is preferred
- Session/state can be externalized (Redis/DB/JWT)

## 6. Stateless vs. Stateful

- Workload requires locality/ordering/affinity
- Throughput depends on partition ownership
- Data must live with compute (stream processing, brokers, DB nodes)

**Staff-level rule:** Default application tier to stateless unless there is a clear performance or correctness reason for statefulness.

## 5. Trade-offs & Common Mistakes

| Mistake                                                              | Why it's wrong                                        |
|----------------------------------------------------------------------|-------------------------------------------------------|
| Calling a service stateless while storing session in local memory    | Breaks on scale-out and restarts                      |
| Forcing everything stateless, including workloads that need affinity | Causes high remote-read latency and poor performance  |
| Sticky sessions as first solution                                    | Uneven load, fragile failover, harder autoscaling     |
| Ignoring state migration plan in stateful systems                    | Rebalancing becomes a production incident             |
| No replication strategy for stateful nodes                           | Single node failure leads to data loss or long outage |

## 6. Key Numbers

| Metric                                      | Value                            |
|---------------------------------------------|----------------------------------|
| Stateless scale-out time (Kubernetes pod)   | ~5-30 seconds                    |
| Stateful failover time (DB leader election) | ~5-60 seconds                    |
| In-memory lookup latency                    | microseconds to low milliseconds |
| Remote cache lookup (same region)           | ~0.5-3ms                         |
| DB read/write                               | ~2-20ms                          |
| Stateful repartitioning (large data)        | minutes to hours                 |

## 7. Scenario

Designing a real-time food delivery platform:

Stateless tier:

- API servers (search restaurants, place order, track status)
- Auth service (validates JWT, no local state)
- Notification dispatcher (reads template from DB, sends, forgets)

Scaling: autoscale pods based on CPU/QPS

Failure: LB routes around dead pods, zero impact

Stateful tier:

- Order DB (PostgreSQL with read replicas)
- Live courier tracking (WebSocket gateways with shard affinity by city)
- Event stream (Kafka brokers owning partitions for order state transitions)
- Redis cluster (session/rate-limit state, partitioned by key hash)

Scaling: add nodes + rebalance partitions

Failure: replica promotion, brief blip during leader election

Why mixed model works:

- Stateless where elasticity matters
- Stateful where continuity, ordering, or locality matters

## 8. Practical Question

**Q:** You're designing a chat application with 5 million concurrent users. Should chat service instances be stateless or stateful?

**A:** Use a hybrid architecture.

Layer 1: Stateless REST APIs

- Login, profile, message history fetch, search
- Easy autoscaling and rolling deploys
- Any instance handles any request
- Source of truth: database

Layer 2: Stateful WebSocket Gateways

- Maintain active connections in memory
- Use shard affinity (e.g., `hash(user_id) % N gateways`)
- Each gateway knows which users are connected to it
- Keeps presence state (online/offline/typing) in memory

Layer 3: Externalized Durable State

- Messages persisted to durable store (Cassandra/PostgreSQL)
- Presence metadata replicated to Redis (source of truth beyond single gateway)
- Event bus (Kafka) for cross-gateway message fan-out

Failure handling:

- If a stateful gateway dies:
  - Phase 1: Clients detect disconnect (heartbeat timeout ~5s)
  - Phase 2: Clients reconnect to another gateway (via LB)
  - Phase 3: New gateway restores presence from Redis
  - Phase 4: User is back online within ~10 seconds
- Accept short-lived reconnect spike

fully stateless backend layer is usually in-memory (constant remote lookup for every message). A fully stateful everything architecture is hard to scale and operate. Hybrid gives both elasticity and real-time performance.

# Topic 4: Rebalancing & Resharding

## 1. Intuition

You run a warehouse with 4 storage rooms. Each room holds packages for a range of zip codes:

```
Room 1: 00000-24999
Room 2: 25000-49999
Room 3: 50000-74999
Room 4: 75000-99999
```

Business booms. Room 2 (NYC, NJ, PA — dense population) is overflowing while Room 4 (rural areas) is half empty. You have two options:

**Rebalancing:** Move some of Room 2's shelves into Room 4. Reassign zip code boundaries so the load evens out. Same number of rooms, different distribution.

**Resharding:** Build a 5th and 6th room, then redistribute packages across all 6 rooms. More rooms, new distribution.

The hard part isn't deciding to do it — it's **moving packages while the warehouse is still open and accepting deliveries**. You can't shut down and reorganize. Customers are walking in. That's the real challenge.

## 2. Reality

### Definitions

**Rebalancing:** Redistributing data or load across **existing** shards/nodes so that each node carries a roughly equal share. The number of shards may stay the same.

**Resharding:** Changing the **number of shards** (usually increasing) and redistributing all data under a new partitioning scheme. Resharding always involves rebalancing, but rebalancing doesn't always involve resharding.

### Why it happens

| Trigger            | Example                               |
|--------------------|---------------------------------------|
| Uneven data growth | One shard has 500GB, others have 50GB |

|                      |                                                     |
|----------------------|-----------------------------------------------------|
| Hot partition        | Celebrity user's shard is saturated at 95% CPU      |
| Node failure         | A node dies, its shard must move to surviving nodes |
| Capacity expansion   | Traffic doubled, you need more shards               |
| Capacity contraction | Traffic dropped, you want fewer nodes to save cost  |

## Pros (of doing it well)

- Even load distribution → predictable latency
- Better resource utilization → lower cost
- Unlocks further horizontal scaling
- Handles organic data growth gracefully

## Cons (the pain)

- Data movement consumes network bandwidth and disk I/O
- Risk of temporary inconsistency or unavailability during migration
- Routing layer must be updated atomically (or progressively)
- Complex coordination — especially under live traffic
- Can cause cache invalidation storms

## 3. How It Works

There are fundamentally **three strategies** the industry uses.

### Strategy A: Fixed Partition Count (Pre-split)

**Setup:** Create far more partitions than nodes upfront. Example: 256 partitions spread across 4 nodes (64 each).

Initial layout:

```
Node 1: partitions [0, 1, 2, ... 63]      → 64 partitions
Node 2: partitions [64, 65, 66, ... 127]   → 64 partitions
Node 3: partitions [128, 129, 130, ... 191] → 64 partitions
Node 4: partitions [192, 193, 194, ... 255] → 64 partitions
```

Routing rule:

```
partition_id = hash(key) % 256
node = lookup(partition_id) ← from metadata table
```

### Rebalancing when Node 5 is added:

Step 1: PLAN

- Coordinator calculates: 256 partitions / 5 nodes = ~51 each
- Each existing node should give away ~13 partitions

- Pick partitions to move based on size/load (not random)

Step 2: MIGRATE (per partition, e.g., partition 51 from Node 1 → Node 5)

#### Phase 1: COPY

- Node 5 starts background streaming of partition 51 data from Node 1
- Reads for partition 51: still served by Node 1
- Writes for partition 51: still go to Node 1
- Node 5 is catching up, not serving traffic yet

#### Phase 2: CATCH-UP

- Node 5 replays any writes that landed on Node 1 during the copy
- Uses a changelog/WAL stream from Node 1
- Gap between Node 1 and Node 5 shrinks to near-zero (sub-second lag)

#### Phase 3: CUTOVER

- Metadata service atomically updates: "partition 51 → Node 5"
- All new reads/writes for partition 51 now route to Node 5
- Node 1 stops accepting requests for partition 51
- Brief blip possible (milliseconds) during routing switch

#### Phase 4: CLEANUP

- Node 1 deletes its local copy of partition 51 data
- Verify checksum/row-count match between Node 1's old copy and Node 5's new copy
- Log migration as complete

After all migrations:

|                                                       |                 |
|-------------------------------------------------------|-----------------|
| Node 1: partitions [0-50]                             | → 51 partitions |
| Node 2: partitions [64-114]                           | → 51 partitions |
| Node 3: partitions [128-178]                          | → 51 partitions |
| Node 4: partitions [192-242]                          | → 51 partitions |
| Node 5: partitions [51-63, 115-127, 179-191, 243-255] | → 52 partitions |

## What if Node 5 crashes mid-migration?

Failure during COPY/CATCH-UP:

- Metadata still says "partition 51 → Node 1"
- Node 1 is still the owner, still serving traffic
- Node 5's partial copy is discarded
- Migration is retried from scratch when Node 5 recovers
- Zero impact on live traffic ← safety of "cutover last"

Failure during CUTOVER:

- If metadata update didn't complete: Node 1 is still owner, retry cutover
- If metadata update completed but Node 5 dies immediately after:
  - Node 5 comes back, reads its local data, resumes serving
  - If Node 5 is permanently dead: metadata service reassigns partition 51 back to Node 1 (or another node), Node 1 still has data until cleanup
- This is why CLEANUP (Phase 4) is always LAST — old owner keeps data until migration is fully verified

**Used by:** Elasticsearch, Cassandra, Riak, Kafka

## Strategy B: Dynamic Partitioning (Split/Merge)

**Setup:** Start with one or a few partitions. When a partition gets too large or too hot, **split it in half**. When



partitions get too small, merge them

Initial state:

Partition A: keys [a-z] → Node 1 (all data on one node)

Partition A grows past threshold (e.g., 10GB):

System triggers auto-split

## How a split works:

Phase 1: DETECT

- Background monitor checks partition sizes every N seconds
- Partition A = 12GB → exceeds 10GB threshold
- Split is triggered

Phase 2: CHOOSE SPLIT POINT

- System samples keys in Partition A
- Finds the median key: "m"
- Split boundary: [a-m] and [n-z]

Phase 3: CREATE NEW PARTITIONS

- Partition A1 created: keys [a-m] → stays on Node 1
- Partition A2 created: keys [n-z] → assigned to Node 2

Phase 4: COPY DATA FOR A2

- Node 2 starts receiving all keys [n-z] from Node 1
- Reads for [n-z]: still served by Node 1 during copy
- Writes for [n-z]: go to Node 1, replicated to Node 2 via stream

Phase 5: CATCH-UP

- Node 2 replays recent writes for [n-z]
- Lag approaches zero

Phase 6: CUTOVER

- Metadata atomically updated:
  - Partition A [a-z] → Node 1 ← DELETED
  - Partition A1 [a-m] → Node 1 ← NEW
  - Partition A2 [n-z] → Node 2 ← NEW
- Routing layer now sends [a-m] to Node 1, [n-z] to Node 2

Phase 7: CLEANUP

- Node 1 deletes keys [n-z] (now owned by Node 2)
- Checksums verified

## Subsequent growth:

Time 0: [a—————z] → Node 1

Time 1: [a————m] → Node 1      [n————z] → Node 2  
(split at median)

Time 2: [a——f] → Node 1      [g——m] → Node 3      [n————z] → Node 2  
(A1 split again)

Time 3: [a——f] → Node 1      [g——m] → Node 3      [n——t] → Node 2      [u——z] → Node 4  
(A2 split)

Result: 4 partitions across 4 nodes, automatically adapted to data growth

## Merging:

Condition: Two adjacent partitions both drop below 2GB

Partition X [a-f] = 1.5GB on Node 1

Partition Y [g-m] = 1.2GB on Node 3

Phase 1: COPY

- Node 1 starts receiving Partition Y data from Node 3
- Reads for [g-m]: still served by Node 3

Phase 2: CATCH-UP

- Node 1 replays recent writes for [g-m]

Phase 3: CUTOVER

- Metadata update: Partition XY [a-m] → Node 1

Phase 4: CLEANUP

- Node 3 deletes Partition Y data

**Used by:** HBase, MongoDB (auto-split), CockroachDB, TiKV/TiDB

## Strategy C: Consistent Hashing (Virtual Nodes)

**Setup:** Each physical node owns multiple positions (virtual nodes / vnodes) on a hash ring.

Hash Ring (0 to  $2^{32}$ ):

Node A vnodes: positions 1000, 45000, 82000

Node B vnodes: positions 12000, 56000, 91000

Node C vnodes: positions 23000, 67000, 99000

Routing:

key\_hash = hash("user\_123") = 7500

Walk clockwise on ring → next vnode = 12000 → owned by Node B

So "user\_123" lives on Node B

## Rebalancing when Node D joins:

Phase 1: ASSIGN VNODES

- Node D gets vnode positions: 8000, 50000, 75000
- These are inserted into the ring

Phase 2: IDENTIFY AFFECTED KEY RANGES

For vnode 8000 (Node D):

Previous ring: keys 1001-12000 → Node B (vnode 12000)

New ring: keys 1001-8000 → Node D (vnode 8000)

keys 8001-12000 → Node B (still)

Only keys in range 1001-8000 need to move from Node B → Node D

```
For vnode 50000 (Node D):
  Previous: keys 45001-56000 → Node B (vnode 56000)
  New:      keys 45001-50000 → Node D
           keys 50001-56000 → Node B (still)

  Only keys 45001-50000 move

(Same logic for vnode 75000)
```

Phase 3: STREAM DATA (per affected range, e.g., keys 1001-8000)

Phase 3a: COPY

- Node D requests all keys in range 1001-8000 from Node B
- Node B streams them over the network
- Reads: still served by Node B
- Writes: go to Node B, forwarded/replicated to Node D

Phase 3b: CATCH-UP

- Node D replays recent writes from Node B's changelog
- Lag drops to near-zero

Phase 3c: CUTOVER

- Ring metadata updated: Node D now owns vnode 8000
- Reads/writes for keys 1001-8000 route to Node D
- Node B stops serving that range

Phase 3d: CLEANUP

- Node B deletes keys 1001-8000 from local storage
- Checksums verified

Phase 4: REPEAT for each affected vnode range

Total data moved: only  $\sim 1/N$  of all keys (where  $N$  = new total node count)

## What if a node LEAVES (failure or decommission)?

Node C dies. Node C owned vnodes: 23000, 67000, 99000

For vnode 23000:

Keys that were going to Node C (range 12001-23000)

now walk clockwise past 23000 → next vnode = 45000 → Node A

Node A must pick up those keys.

If replication factor = 3, replicas already exist on other nodes.  
Promotion is instant – replica becomes primary.

If no replicas → data loss for that range until recovery from backup.

Ring self-heals:

Before: ... → Node B(12000) → Node C(23000) → Node A(45000) → ...  
After: ... → Node B(12000) → Node A(45000) → ...

Node A now owns a larger key range temporarily.  
System can then rebalance by assigning C's old vnodes to new nodes.

**Used by:** DynamoDB, Cassandra, Riak, Memcached (client-side)

## Why $\text{hash}(\text{key}) \% N$ Is Catastrophic for Rebalancing

Setup: 4 nodes,  $\text{routing} = \text{hash}(\text{key}) \% 4$

```
hash("alice")    = 14  → 14 % 4 = 2 → Node 2
hash("bob")      = 7   → 7  % 4 = 3 → Node 3
hash("charlie")  = 22  → 22 % 4 = 2 → Node 2
hash("dave")     = 9   → 9  % 4 = 1 → Node 1
```

Add Node 5:  $\text{routing} = \text{hash}(\text{key}) \% 5$

```
hash("alice")    = 14  → 14 % 5 = 4 → Node 4  ← MOVED
hash("bob")      = 7   → 7  % 5 = 2 → Node 2  ← MOVED
hash("charlie")  = 22  → 22 % 5 = 2 → Node 2  ← stayed
hash("dave")     = 9   → 9  % 5 = 4 → Node 4  ← MOVED
```

Result: 3 out of 4 keys moved = 75% remapped

At scale:  $\sim(N-1)/N$  keys must move. With 100 nodes → 99% of data remapped.

Compare consistent hashing:  $\sim 1/N$  keys move = 1% with 100 nodes.

## 4. When to Rely on It in SD

It comes up at these moments:

1. **After you propose sharding:** Interviewer asks "What happens when one shard gets too big?" — that's rebalancing.
2. **After you estimate scale:** "You said 4 shards. What happens at 10x growth?" — that's resharding.
3. **When discussing hot keys:** "User X generates 50% of traffic on one shard" — that's rebalancing to split the hot shard.
4. **When discussing node failure:** "Node 3 dies. Where does its data go?" — that's emergency rebalancing.
5. **Cost optimization:** "Traffic dropped 60% after holiday. Can we scale down?" — that's rebalancing to fewer nodes.

## 5. Trade-offs & Common Mistakes

| Mistake                                                   | Why it's wrong                                                                                        |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Using $\text{hash}(\text{key}) \% N$ for shard assignment | Adding one node remaps $\sim(N-1)/N$ keys. Catastrophic. Use consistent hashing or fixed partitions.  |
| No pre-planning for rebalancing                           | "We'll figure it out later." By then you're in a production fire.                                     |
| Rebalancing during peak traffic                           | Data migration competes for I/O with user traffic. Schedule off-peak or throttle migration bandwidth. |

|                                        |                                                                                                                   |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Moving too many partitions at once     | Thundering herd — massive bandwidth spike, cache misses, latency spikes. Move incrementally.                      |
| Not testing the cutover path           | The metadata switch is the riskiest moment. Test it. Have a rollback plan.                                        |
| Running cleanup before verification    | If you delete old data before confirming new owner has it, you lose data. Always verify-then-cleanup.             |
| Ignoring the routing layer propagation | Data moved but clients still route to old node = errors. Routing updates must propagate before or during cutover. |

## 6. Key Numbers

| Metric                                        | Typical value                         |
|-----------------------------------------------|---------------------------------------|
| Consistent hashing: keys moved on add/remove  | ~1/N of total keys                    |
| $\text{hash} \% N$ : keys moved on add/remove | ~(N-1)/N — almost everything          |
| Partition migration speed (same DC, NVMe)     | 100–500 MB/s per stream               |
| Time to migrate 100GB partition (same DC)     | ~3–15 minutes depending on throttling |
| Cross-region partition migration (100GB)      | 30 min to hours                       |
| Typical pre-split partition count (Kafka)     | 3x to 10x expected node count         |
| CockroachDB default range size                | 512 MB (auto-splits above this)       |
| Rebalancing bandwidth cap (common default)    | 50–200 MB/s per node                  |

## 7. Scenario

### Designing a global URL shortener (like bit.ly).

Initial design: 8 shards using consistent hashing with 256 vnodes per node.

6 months later: data 2TB → 20TB, QPS 50K → 500K, 3 shards near capacity.

Solution:

Step 1: SHORT TERM — rebalance within existing nodes

- Identify which vnodes on overloaded nodes can move to underloaded ones
- Stream affected key ranges:
  - Phase 1: COPY → background stream, old node still serves reads
  - Phase 2: CATCH-UP → replay writes, gap → near-zero
  - Phase 3: CUTOVER → metadata update, new node now owns those vnodes
  - Phase 4: CLEANUP → old node deletes migrated ranges, checksums verified
- Result: load evens out without adding hardware

Step 2: MEDIUM TERM — reshard by adding nodes

- Add 24 new nodes (8 → 32 total)
- Each new node gets vnodes on the ring
- Consistent hashing: only ~75% of keys move (spread across all new nodes)
- Migration throttled at 150 MB/s per node to protect user traffic
- Migrations run in parallel batches (4-8 partitions at a time)

Step 3: ROUTING

- API servers fetch partition→node map from metadata service (etcd/ZooKeeper)
- Map cached locally with 5s TTL
- On cutover, metadata updates → API servers pick up new mapping within seconds
- During catch-up window: stale routes hit old owner, which forwards or returns redirect

## 8. Practical Question

**Q:** You're running a chat application on 10 database shards (consistent hashing, hash on `chat_room_id`). A K-pop band announces a live fan event and 2 million users flood into one chat room. That room's shard is at 100% CPU while other shards sit at 20%. How do you handle this?

**A:**

**Critical insight first:** This is a **hot key problem**, not a general rebalancing problem. Rebalancing moves entire partitions — but this is one key (`chat_room_id = "kpop_live_event"`) melting one shard. Moving the partition to a different node just moves the fire.

IMMEDIATE (minutes):

Step 1: Vertically scale the hot shard's node

- Move to a bigger instance (more CPU/RAM)
- Buys time, does not fix root cause

Step 2: Add read replicas for the hot shard

- Route read traffic (message history, member list) to replicas
- Write path stays on primary
- Result: read QPS distributed across replicas

Step 3: Cache aggressively

- Recent messages → Redis
- Member list → Redis with 2s TTL
- Fan-out reads from cache, not DB

SHORT TERM (hours):

Step 4: Buffer writes

- Put a message queue in front of the DB shard
- Consumers batch-insert messages (100 at a time instead of 1 at a time)
- Reduces write IOPS by ~100x

LONG TERM (architecture):

Step 5: Sub-sharding for hot keys

- Don't just hash on `chat_room_id`
- For rooms above a member threshold:

```
shard_key = hash(chat_room_id + message_time_bucket)
- One room's messages now spread across multiple shards
- Reads: scatter-gather across sub-shards (acceptable for chat history)
- Writes: each sub-shard handles a time window
```

#### Step 6: Proactive hot key detection

- Monitor per-shard CPU/QPS in real time
- When a single key exceeds threshold QPS → auto-enable sub-sharding
- Alert ops if a shard crosses 70% utilization

#### Step 7: Separate the read path entirely

- Write path: Room → Kafka partition → DB shard (write-optimized)
- Read path: Kafka → Read-optimized store (Elasticsearch or Redis Streams)
- Reads never hit the primary DB shard at all

**The one-liner for interviews:** Rebalancing solves uneven distribution across shards. It does NOT solve hot keys. Hot keys need **key-level splitting, caching, or read-path separation** — not shard-level movement.

## Bonus: Shard Affinity (Quick Reference)

**Definition:** Routing requests with the same partition key to the same shard to improve locality, cache efficiency, and consistency of processing.

**How:** Hash the partition key (e.g., `user_id`) and deterministically route to the same shard. Any router can compute the correct shard — no central lookup needed.

#### Why it helps:

- Better cache hit rate (hot data stays warm on one node)
- Lower latency (no cross-shard context fetching)
- Easier ordering (events for same entity stay in one partition)
- Less cross-shard chatter

**Downside:** Can cause hot spots if one key becomes extremely popular. Mitigate with sub-sharding or load-aware routing.

## Topic 5: Hot Key Problem

### 1. Intuition

Imagine a post office with 10 counters. Each counter handles mail for a specific set of zip codes. On a normal day, each counter gets roughly equal traffic.

Then Taylor Swift announces a concert and 500,000 fans rush to buy tickets. All those orders ship to the





|       |                                         |                      |
|-------|-----------------------------------------|----------------------|
| Kafka | One customer_id producing 90% of events | Skewed partition key |
|-------|-----------------------------------------|----------------------|

### Pros of recognizing it:

- Targeted fix instead of throwing hardware at the whole system
- Often solvable with caching or key-splitting without re-architecting
- Measurable — you can detect it with per-key metrics

### Cons (why it's hard):

- Unpredictable — you often can't know which key will go hot in advance
- Standard sharding/hashing doesn't protect you — it evenly distributes keys, but one key always maps to one shard
- Fixes introduce read complexity (scatter-gather) or consistency trade-offs
- Can cascade: hot key → node overload → increased latency for ALL keys on that node → user-visible degradation

## 3. How It Works

### 3A. How a Hot Key Forms

System: 8-shard database, consistent hashing on item\_id

Normal state:

```
Shard 1: 12,500 QPS across 100K keys
Shard 2: 12,800 QPS across 105K keys
Shard 3: 12,200 QPS across 98K keys
...
Shard 8: 12,600 QPS across 102K keys
Total: ~100K QPS evenly distributed
```

Flash sale starts for item\_id = "DEAL-777":

```
hash("DEAL-777") % ring → Shard 5
```

```
Shard 1: 12,500 QPS
Shard 2: 12,800 QPS
Shard 3: 12,200 QPS
Shard 4: 12,300 QPS
Shard 5: 12,500 QPS (normal) + 500,000 QPS (DEAL-777) = 512,500 QPS ← DEAD
Shard 6: 12,400 QPS
Shard 7: 12,100 QPS
Shard 8: 12,600 QPS
```

Shard 5 consequences:

```
Phase 1: CPU hits 100%, response times spike from 5ms → 2000ms
Phase 2: Connection pool exhausted, new requests start timing out
Phase 3: Health check fails, load balancer marks Shard 5 as unhealthy
Phase 4: ALL keys on Shard 5 (not just DEAL-777) become unavailable
Phase 5: Upstream services hitting Shard 5 start timing out → cascading failure
```

The key insight: **one hot key doesn't just affect itself — it kills every other key on the same shard.**

## 3B. Hot Reads vs Hot Writes

These are fundamentally different problems requiring different solutions:

HOT READS (most common):

- Same key read millions of times
- Example: viral tweet, trending product, live score
- The shard is overwhelmed by SELECT/GET operations
- DB CPU, memory, and connection pool saturated

Fix approach: Add layers BETWEEN the reader and the shard

→ Caching, read replicas, CDN

HOT WRITES (harder):

- Same key written/updated at extreme rates
- Example: like counter on viral post, inventory decrement in flash sale, real-time vote tally
- The shard is overwhelmed by UPDATE/INSERT operations
- Row-level locks, WAL throughput, disk I/O saturated

Fix approach: Reduce writes hitting the single key

→ Write buffering, counter splitting, async aggregation

## 3C. Solution Toolkit (The Mechanics)

### Solution 1: Read-Through Cache (for hot reads)

Before (hot reads hitting DB directly):

```
Client → API Server → DB Shard 5 (item_id = "DEAL-777")
Client → API Server → DB Shard 5 (item_id = "DEAL-777")
Client → API Server → DB Shard 5 (item_id = "DEAL-777")
... 500K times per second
```

After (cache absorbs hot reads):

```
Client → API Server → Cache (Redis/Memcached)
                        ↓ (cache hit? return immediately)
                        ↓ (cache miss? fetch from DB, store in cache)
                        → DB Shard 5
```

Phase 1: FIRST REQUEST (cache miss)

- API checks cache for key "item:DEAL-777" → miss
- API reads from DB Shard 5 → gets product data
- API writes to cache: SET "item:DEAL-777" {data} TTL=10s
- Returns to client

Phase 2: SUBSEQUENT REQUESTS (cache hit)

- API checks cache for "item:DEAL-777" → hit
- Returns cached data directly
- DB Shard 5 never touched
- Cache handles 499,999 of the 500K requests

Result:

```
Cache: 499,999 QPS for this key (~1ms response)
DB Shard 5: 1 QPS for this key (one miss every 10s TTL refresh)
Shard 5 is no longer hot
```

## ~~But what if the cache node itself becomes hot?~~

Problem: All 500K reads for "item:DEAL-777" go to ONE Redis node  
(because the key hashes to one node in the Redis cluster)

Solution: LOCAL CACHE on each API server

```
Client → API Server (local in-memory cache, e.g., Guava/Caffeine)
      ↓ (local hit? return instantly — no network call)
      ↓ (local miss?)
      → Redis (distributed cache)
        ↓ (Redis hit? return)
        ↓ (Redis miss?)
        → DB Shard 5
```

With 20 API servers, each caching locally with 5s TTL:

- Each server holds its own copy of "DEAL-777" data
- 500K QPS distributed across 20 servers → 25K per server
- Each server hits Redis at most once every 5 seconds
- Redis hit rate: ~99.99%
- DB hit rate: once every 10s

Cache hierarchy:

- Layer 1: L1 local cache (in-process, microsecond access)
- Layer 2: L2 distributed cache (Redis, ~1ms access)
- Layer 3: Database (last resort, ~5-20ms)

## Solution 2: Read Replicas (for hot reads on DB)

Before:

All reads and writes for Shard 5 → one primary node

After:

Writes → Primary (Shard 5 leader)  
Reads → Distributed across N read replicas

Phase 1: SETUP REPLICAS

- Create 3 read replicas for Shard 5
- Replicas stream WAL from primary in real-time
- Replication lag: ~10-100ms (async) or 0ms (sync, slower writes)

Phase 2: ROUTE READS

- Load balancer sends read traffic to replicas round-robin
- API: if operation == READ → route to replica  
if operation == WRITE → route to primary

Phase 3: RESULT

- Primary: handles only writes (~2K QPS for DEAL-777 stock updates)
- Replica 1: handles 166K read QPS
- Replica 2: handles 166K read QPS
- Replica 3: handles 168K read QPS
- Total read capacity: 3x original

Trade-off: Replicas may serve slightly stale data (replication lag)

Acceptable for: product page views, tweet reads, profile lookups

Not acceptable for: inventory count (user sees "in stock" but it's sold out)

## ~~Solution 3: Key Splitting / Scatter-Gather (for hot reads & writes)~~

Problem: `item_id = "DEAL-777"` always maps to one shard

Solution: Split the key into N sub-keys

Instead of:

`key = "DEAL-777" → shard 5`

Use:

`key = "DEAL-777:0" → shard 2`

`key = "DEAL-777:1" → shard 5`

`key = "DEAL-777:2" → shard 7`

`key = "DEAL-777:3" → shard 1`

For WRITES (e.g., incrementing a like counter):

Phase 1: Client picks a random sub-key

`sub_key = "DEAL-777:" + random(0, 3) → e.g., "DEAL-777:2"`

Phase 2: Increment on that sub-key's shard

`INCR "DEAL-777:2" → shard 7`

Phase 3: Writes are now distributed across 4 shards

`500K write QPS / 4 = 125K per shard (manageable)`

For READS (e.g., reading the total like count):

Phase 1: SCATTER – read all sub-keys in parallel

`GET "DEAL-777:0" → shard 2 → returns 125,432`

`GET "DEAL-777:1" → shard 5 → returns 124,891`

`GET "DEAL-777:2" → shard 7 → returns 125,103`

`GET "DEAL-777:3" → shard 1 → returns 124,574`

Phase 2: GATHER – sum them up

`Total likes = 125,432 + 124,891 + 125,103 + 124,574 = 500,000`

Phase 3: Return to client

`Response: {likes: 500000}`

Trade-off: Reads are now 4 parallel calls instead of 1 (higher latency)

Single key read: ~2ms

Scatter-gather (4 parallel): ~5-8ms (limited by slowest shard)

Acceptable trade-off for eliminating the hot key

## Solution 4: Write Buffering / Async Aggregation (for hot writes)

Problem: 500K/s writes to one counter (like count, view count, inventory)

Solution: Buffer writes in memory or a queue, flush periodically

Phase 1: BUFFER

- API server receives "like" request

- Instead of: `UPDATE posts SET likes = likes + 1 WHERE id = 'DEAL-777'`

- Does: increment local in-memory counter for DEAL-777

- Each API server buffers its own count

Phase 2: FLUSH (every 1 second or every 1000 increments)

- API Server 1: accumulated 25,000 likes →

```
UPDATE posts SET likes = likes + 25000 WHERE id = 'DEAL-777'
- API Server 2: accumulated 24,800 likes →
  UPDATE posts SET likes = likes + 24800 WHERE id = 'DEAL-777'
- ... (20 servers flushing)
```

#### Phase 3: RESULT

```
Before: 500,000 DB writes/second to one row
After:  20 DB writes/second to one row (one per API server per second)
Reduction: 25,000x fewer writes
```

#### Trade-off:

- Like count is eventually consistent (up to 1 second stale)
- If an API server crashes, buffered counts are lost (acceptable for likes; not acceptable for payments)
- More complex client code (buffer management, flush logic)

## Solution 5: CDN / Edge Cache (for hot reads of static/semi-static data)

Problem: Viral product image or video being fetched 10M times

Solution: Push to CDN – requests never reach your origin servers

#### Phase 1: FIRST REQUEST

```
Client (Tokyo) → CDN PoP (Tokyo) → cache miss → Origin Server → DB
CDN caches the response with TTL = 60s
```

#### Phase 2: ALL SUBSEQUENT REQUESTS FROM THAT REGION

```
Client (Tokyo) → CDN PoP (Tokyo) → cache hit → return immediately
Origin server: 0 requests
DB: 0 requests
```

#### Phase 3: SCALE

```
CDN has 200+ PoPs worldwide
Each PoP caches independently
10M requests/hour served entirely from edge
Origin sees ~200 requests/hour (one per PoP per TTL window)
```

Works for: images, videos, product listings, public profiles, API responses  
Doesn't work for: personalized data, real-time counters, mutable state

## 4. When to Rely on It in SD

The hot key problem surfaces at these interview moments:

1. **After sharding discussion:** "You've sharded by user\_id. What if one user generates 90% of events?"
2. **Flash sale / viral content:** "How does your system handle a product going viral?"
3. **Counter at scale:** "How do you track likes/views for a post with 100M likes?"
4. **Cache design:** "What if one cache key gets 1M reads/second?"
5. **After you say 'we shard on X':** The interviewer probes: "What about skewed distribution?"

**The staff-level signal:** Proactively mention hot keys before the interviewer asks. When you draw your

sharding strategy, say: "This works for uniform distribution, but we need to handle hot keys separately, here's how..." That's the architect mindset.

## 5. Trade-offs & Common Mistakes

| Mistake                                        | Why it's wrong                                                                                                                        |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Thinking rebalancing/resharding fixes hot keys | Moving a hot key's shard to a bigger node just moves the fire. The key itself needs splitting or caching.                             |
| No hot key detection                           | You find out from a production outage instead of monitoring. Track per-key QPS or at least per-shard QPS.                             |
| Over-caching with long TTL                     | Stale data becomes a correctness issue. Flash sale item shows "in stock" for 60s after selling out.                                   |
| Key splitting with too many sub-keys           | 1000 sub-keys for a counter means 1000-way scatter-gather on every read. Start with 4-16 sub-keys.                                    |
| Buffering writes for critical data             | Buffered like counts are fine. Buffered payment amounts are not. Match the technique to the data's importance.                        |
| Ignoring the blast radius                      | A hot key doesn't just affect itself — it degrades ALL other keys on the same shard. Fix the hot key to save the neighbors.           |
| Assuming uniform distribution                  | Real-world data is almost always Zipfian (power-law). A small number of keys will always get disproportionate traffic. Design for it. |

## 6. Key Numbers

| Metric                                      | Typical value                               |
|---------------------------------------------|---------------------------------------------|
| Redis single-key read throughput (one node) | ~100K-300K ops/sec                          |
| Memcached single-key read throughput        | ~200K-500K ops/sec                          |
| Single DB row read throughput (PostgreSQL)  | ~10K-50K QPS (depends on row size, indexes) |
| Single DB row write throughput              | ~1K-10K QPS (lock contention, WAL)          |
| CDN edge cache hit latency                  | ~1-5ms                                      |
| L1 local cache hit latency (in-process)     | ~1-100 microseconds                         |
| L2 distributed cache hit latency (Redis)    | ~0.5-3ms                                    |

|                                     |                                    |
|-------------------------------------|------------------------------------|
| Write buffering reduction factor    | 1000x-50000x fewer DB writes       |
| Typical sub-key count for splitting | 4-16 per hot key                   |
| Zipf's law in practice              | Top 1% of keys → 20-50% of traffic |

## 7. Scenario

### Designing a real-time election results dashboard.

On election night, one endpoint — `/results/presidential` — gets 50M requests per minute. There are also 500 other result pages (state races, local races) at normal traffic.

Architecture:

Layer 1: CDN (Cloudflare/CloudFront)

- Cache `/results/presidential` with TTL = 5 seconds
- 50M requests/min → CDN serves 49.99M from edge
- Origin sees ~12 requests/min per PoP × 200 PoPs = ~2400 req/min
- Acceptable staleness: 5 seconds (election results don't change faster)

Layer 2: API servers (stateless)

- L1 local cache with 2s TTL for `/results/presidential`
- On cache miss → read from Redis

Layer 3: Redis (distributed cache)

- Key `"results:presidential"` replicated to 3 Redis replicas
- Read from replicas (round-robin)
- TTL = 5s

Layer 4: Database

- Results table, updated every ~30 seconds as precincts report
- On update → invalidate Redis key → CDN purge
- DB sees ~2 writes/minute for presidential results
- Zero hot key problem at DB layer

Write path (when results update):

- Phase 1: Election authority pushes update → API receives it
- Phase 2: API writes to DB
- Phase 3: API invalidates Redis: `DEL "results:presidential"`
- Phase 4: API sends CDN purge for `/results/presidential`
- Phase 5: Next read triggers cache refill through all layers
- Phase 6: Stale window: ~0-5 seconds (CDN TTL)

## 8. Practical Question

**Q:** You're designing Instagram's like system. A celebrity posts a photo and it receives 2 million likes in 10 minutes. Your like count is stored as a column in a `__CODE0 table, sharded by CODE1__`. The shard for this post is melting. How do you fix this — immediately and long-term?

IMMEDIATE (within minutes – stop the bleeding):

Step 1: Cache the current like count in Redis

- SET "likes:post:CELEB-123" 1847293 TTL=5s
- All read requests (displaying the count on the post) hit Redis, not the DB
- DB read load for this post → near zero

Step 2: Stop writing every like directly to DB

- Route like events to a Kafka topic: "likes-stream"
- Key = post\_id, so all likes for one post go to one partition
- Consumer batches: every 1 second OR every 5000 likes, do ONE update:  
UPDATE posts SET like\_count = like\_count + 5000  
WHERE post\_id = 'CELEB-123'

Before: 200K writes/min → one row

After: 60 writes/min → one row (one batch per second)

SHORT TERM (within hours):

Step 3: Split the like counter

- Instead of one like\_count column, use a like\_counts table:

| post_id   | bucket | count   |
|-----------|--------|---------|
| CELEB-123 | 0      | 461,823 |
| CELEB-123 | 1      | 462,001 |
| CELEB-123 | 2      | 461,145 |
| CELEB-123 | 3      | 462,324 |

- Write: like lands on random bucket (0-3)  
UPDATE like\_counts SET count = count + 1  
WHERE post\_id = 'CELEB-123' AND bucket = random(0,3)
- Read: SUM across buckets → cache the result  
SELECT SUM(count) FROM like\_counts  
WHERE post\_id = 'CELEB-123'  
→ 1,847,293 → cache in Redis with 5s TTL
- 4 buckets × potentially different shards = 4x write throughput

LONG TERM (architecture):

Step 4: Async counter service

- Dedicated counter service backed by a system designed for high-write counters (Redis INCR, or a custom counter store)
- Like event → Kafka → Counter Service → Redis INCR
- Periodically flush Redis counter to DB for persistence
- Read path always hits Redis (never DB for hot counters)

Flow:

User taps like → API → Kafka "likes" topic

Counter consumer → INCR "likes:CELEB-123" in Redis

Background job (every 30s) → read Redis → UPDATE DB

Read: GET "likes:CELEB-123" from Redis → return instantly



#### Step 5: Hot key detection

- Monitor per-post write QPS
- When a post\_id exceeds 10K likes/min → auto-enable:
  - (a) Kafka buffering for writes
  - (b) Redis-backed counter
  - (c) 4-bucket split if sustained
- Normal posts (< 100 likes/min) → direct DB update (simple path)

#### Result:

- DB shard: no longer hot (batched/split writes, cached reads)
- Like counts: eventually consistent (up to 5s stale for display)
- Correctness: total likes are accurate after flush
- User experience: they see count update within seconds (good enough)

**Interview one-liner:** For hot-read keys, cache aggressively (L1 → L2 → CDN). For hot-write keys, buffer and batch writes, or split the key into N sub-keys to distribute writes across shards. Detect hot keys proactively — don't wait for the outage.